executed

September 26, 2026

1 Climate anxiety and flooding in Sylhet: analysis code

This notebook reproduces every number reported in the manuscript from the de-identified survey file that is deposited with it (`ESM_6.xlsx`, sheet `Data`). Running the cells from top to bottom regenerates the reliability statistics, the factor analysis, every table and figure in the paper and its supplement, and the regression diagnostics. Nothing is read from anywhere else and no result is hard-coded.

The deposited file holds the 212 eligible adults. Four of the 216 people who completed the questionnaire were under 18 and were removed before any analysis, so the exclusion is already applied in the file and is not repeated here.

Scoring. The CCAS is scored as the mean of its items, so it stays on the 1 to 5 response scale. The three anxiety bands cut that range into equal thirds: low below 2.33, moderate from 2.33 to below 3.67, high at 3.67 and above. Table S2 uses a lower cut of 3.5 for the "% high anxiety" column, which is stated in the table footnote.

Regression. Outcome and predictors are all z-scored before estimation, including the two binary predictors, so the coefficients in Table 2 and Table 3 are standardized betas. Standard errors are HC3. All three blocks are fitted on the same 196 respondents who have complete data on every model variable, otherwise the change in R-squared between blocks would not be comparable.

Numbering. Tables 1 to 3 and Figures 1 to 3 are the items printed in the paper. Items labelled S below are in Online Resource 8. The sections here run in analysis order, so a section may produce a supplementary item.

Requirements. pandas, numpy, scipy, matplotlib, openpyxl. The HC3 standard errors, the Benjamini-Hochberg correction, the KMO statistic and the varimax rotation are written out below rather than imported, so the notebook does not depend on statsmodels or factor_analyzer.

[1]:
```
import io
import os

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from scipy import stats

pd.set_option('display.width', 140)
```

```
pd.set_option('display.max_columns', 60)
pd.set_option('display.max_rows', 100)

DATA = 'ESM_6.xlsx'
SEED = 42                       # used only by the permutation test in section 12
ALPHA = 0.05

import scipy
print('pandas %s | numpy %s | scipy %s' % (pd.__version__, np.__version__,␣
 ↪scipy.__version__))
```

```
pandas 2.3.3 | numpy 2.2.6 | scipy 1.15.3
```

1.1 1. Load

[2]:
```
raw = pd.read_excel(DATA, sheet_name='Data')

# a few free-text and category cells came out of KoboToolbox with stray leading␣
 ↪or
# trailing spaces (Q21, Q38 and Q40 in particular), which would otherwise split␣
 ↪a
# category in two. isinstance is used rather than a dtype test so that the␣
 ↪strip also
# happens under the newer pandas string dtype, where text columns are no longer␣
 ↪object
def strip_text(s):
    return s.map(lambda v: v.strip() if isinstance(v, str) else v)

raw = raw.apply(strip_text)
assert not raw.apply(lambda s: s.map(lambda v: isinstance(v, str) and v != v.
 ↪strip())).any().any()

print('Respondents :', len(raw))
print('Variables   :', raw.shape[1])
print('Age range   : %d to %d' % (raw['Age'].min(), raw['Age'].max()))
raw.head(3)
```

```
Respondents : 212
Variables   : 66
Age range   : 18 to 76
```

[2]:
```
  ResponseID  Q7_Loss_HouseProperty  Q7_Loss_CropLoss  Q7_Loss_Livestock
Q7_Loss_Infrastructure  Q7_Loss_Vehicles  Q7_Loss_HealthRisks  \
0       R001                     1.0               0.0                 0.0
0.0                0.0                   0.0
1       R002                     1.0               0.0                 0.0
0.0                0.0                   0.0
```

```
2       R003                    1.0               1.0                0.0
0.0              0.0                  1.0

   Q7_Loss_Utilities  Q7_Loss_IncomeLoss  Q7_Loss_Sanitation  Q7_Loss_Education
Q7_Loss_PsychStress  Q7_Loss_SafetyRisks  Q7_Loss_Debt  \
0                0.0                 0.0                 0.0                0.0
0.0                  0.0          0.0
1                0.0                 0.0                 0.0                0.0
0.0                  0.0          0.0
2                1.0                 1.0                 1.0                0.0
1.0                  0.0          0.0

   Q7_Loss_SocialInsecurity  Q7_Loss_Others CCAS_Q8_Concentrate  CCAS_Q9_Sleep
CCAS_Q10_Nightmares CCAS_Q11_Crying CCAS_Q12_HandleBetter  \
0                       0.0             0.0               Often  Almost Always
Often           Often                 Never
1                       0.0             0.0       Almost Always          Often
Often           Often                Rarely
2                       1.0             0.0       Almost Always  Almost Always
Almost Always       Sometimes                Never

  CCAS_Q13_PreferAlone CCAS_Q14_ReflectDeeply CCAS_Q15_QuestionReact
CCAS_Q16_FamilyFriends CCAS_Q17_BalanceDaily CCAS_Q18_WorkStudies  \
0        Almost Always                 Often                 Often
Sometimes                 Often               Often
1                Never          Almost Always                 Never
Often         Almost Always        Almost Always
2                Never          Almost Always              Sometimes
Almost Always             Sometimes                Often

  CCAS_Q19_Potential CCAS_Q20_OthersToldTooMuch  Q26_React_SeekInfo  …
Q32_Resp_NGOs  Q32_Resp_AllOfAbove       Area  \
0      Almost Always                 Sometimes                   0  …
0                   0  Kadamtoli
1      Almost Always                 Sometimes                   0  …
0                   0  Kadamtoli
2      Almost Always                    Often                    0  …
0                   0  Kadamtoli

       Q1_YearsInCity  Q2_ResidenceType               Q3_AreaType
Q4_FloodFrequency  Q5_ExperiencedFlood  \
0  More than 10 years         Own house  Floodplain/Low-lying area
Annually                  Yes
1  More than 10 years         Own house  Floodplain/Low-lying area
Annually                  Yes
2  More than 10 years         Own house  Floodplain/Low-lying area
Annually                  Yes
```

```
                                   Q6_FloodSeverity Q21_CommitteeExists
Q22_MainSupport Q23_CooperationFreq  \
0  Moderate (Damaged homes or roads, but not dest…            Disagree
Family members              Rarely
1  Moderate (Damaged homes or roads, but not dest…            Disagree
Government agencies           Sometimes
2  Moderate (Damaged homes or roads, but not dest…            Disagree
Family members               Never

  Q24_CooperationReducesImpact              Q25_TalkTo Q27_GovtImproved
Q28_AuthoritiesRespond Q29_SatisfiedPreparedness  \
0                     Disagree  Talk to family members         Disagree
Agree                  Disagree
1                     Disagree  Talk to family members         Disagree
Disagree                  Disagree
2                     Disagree  Talk to family members         Disagree
Disagree                  Disagree

  Q30_DrainageEffectiveness Q31_NoticedInitiatives
Q33_Rank1                                  Q33_Rank2  \
0               Ineffective                     Yes  Social support from family
and friends              Strong community cooperation
1               Ineffective                      No            Strong community
cooperation  Better urban planning and infrastructure
2          Very ineffective                      No  Social support from family
and friends  Better urban planning and infrastructure

                               Q33_Rank3
Q33_Rank4                 Q34_Suggestions Age  \
0          Government disaster management  Better urban planning and
infrastructure  Development of Drainage system  48
1  Social support from family and friends            Government disaster
management                  River Dredging  65
2          Government disaster management              Strong community
cooperation                 Cannel Dredging  40

           Q36_Education Q37_Marital  Q38_Occupation Q39_Income Q40_Gender
0    No formal education     Married       Housewife          0     Female
1         Primary school     Married  Business Owner      10000       Male
2  Secondary/SSC/O-Level     Married  Business Owner      20000       Male

[3 rows x 66 columns]
```

1.2 2. Recode into analysis variables

Every ordered category is mapped to the numeric coding used in the paper. Education answers of "Other" cannot be placed on the ordinal scale so they are set to missing, which is why the education correlation in Table 2 is on n = 205 rather than 212.

[3]:
```
likert = {'Never': 1, 'Rarely': 2, 'Sometimes': 3, 'Often': 4, 'Almost Always':␣
 ↪5}

cog_emo = ['CCAS_Q8_Concentrate', 'CCAS_Q9_Sleep', 'CCAS_Q10_Nightmares',␣
 ↪'CCAS_Q11_Crying',
           'CCAS_Q12_HandleBetter', 'CCAS_Q13_PreferAlone',␣
 ↪'CCAS_Q14_ReflectDeeply',
           'CCAS_Q15_QuestionReact']
functional = ['CCAS_Q16_FamilyFriends', 'CCAS_Q17_BalanceDaily',␣
 ↪'CCAS_Q18_WorkStudies',
              'CCAS_Q19_Potential', 'CCAS_Q20_OthersToldTooMuch']
ccas_items = cog_emo + functional

items = raw[ccas_items].apply(lambda c: c.map(likert)).astype(float)
assert items.notna().all().all(), 'unmapped CCAS response found'

d = pd.DataFrame(index=raw.index)
d['ccas_total'] = items.mean(axis=1)
d['ccas_ce'] = items[cog_emo].mean(axis=1)
d['ccas_fn'] = items[functional].mean(axis=1)

LOW_CUT, HIGH_CUT = 1 + 4 / 3, 1 + 8 / 3          # 2.333 and 3.667
d['band'] = pd.cut(d['ccas_total'], [-np.inf, LOW_CUT, HIGH_CUT, np.inf],
                   right=False, labels=['Low', 'Moderate', 'High'])

severity_map = {'Mild (Minor inconvenience)': 1,
                'Moderate (Damaged homes or roads, but not destructive)': 2,
                'Very Severe (Destroyed homes and infrastructure)': 3}
area_map = {'Elevated area': 1, 'Moderately flood-prone': 2, 'Floodplain/
 ↪Low-lying area': 3}
freq_map = {'Rarely': 1, 'Bi-annually': 2, 'Annually': 3}
coop_map = {'Never': 1, 'Rarely': 2, 'Sometimes': 3, 'Often': 4, 'Always': 5}
agree_map = {'Strongly Disagree': 1, 'Disagree': 2, 'Neutral': 3, 'Agree': 4,␣
 ↪'Strongly Agree': 5}
edu_map = {'No formal education': 1, 'Primary school': 2, 'Secondary/SSC/
 ↪O-Level': 3,
           'Higher Secondary/HSC/A-Level': 4, "Bachelor's degree": 5,
           "Master's degree or higher": 6, 'Other': np.nan}
tenure_map = {'1-5 years': 1, '6-10 years': 2, 'More than 10 years': 3}

d['severity'] = raw['Q6_FloodSeverity'].map(severity_map)
d['area_type'] = raw['Q3_AreaType'].map(area_map)
```

```
d['frequency'] = raw['Q4_FloodFrequency'].map(freq_map)
d['tenure'] = raw['Q1_YearsInCity'].map(tenure_map)
d['coop'] = raw['Q23_CooperationFreq'].map(coop_map)
d['coop_reduces'] = raw['Q24_CooperationReducesImpact'].map(agree_map)
d['authorities'] = raw['Q28_AuthoritiesRespond'].map(agree_map)
d['education'] = raw['Q36_Education'].map(edu_map)
d['age'] = pd.to_numeric(raw['Age'], errors='coerce')
d['income'] = pd.to_numeric(raw['Q39_Income'], errors='coerce')

# the loss question is multi-select and stored as one 0/1 column per option;␣
 ↪the free-text
# "Others" bucket is left out of the count
loss_cols = [c for c in raw.columns if c.startswith('Q7_Loss_') and c !=␣
 ↪'Q7_Loss_Others']
losses = raw[loss_cols].apply(pd.to_numeric, errors='coerce').fillna(0).
 ↪astype(int)
d['n_losses'] = losses.sum(axis=1)

d['female'] = (raw['Q40_Gender'] == 'Female').astype(float)
d['confide'] = (raw['Q25_TalkTo'] == 'Talk to friends or neighbors').
 ↪astype(float)
d['area'] = raw['Area']

model_vars = ['age', 'income', 'education', 'female', 'severity', 'n_losses',␣
 ↪'area_type',
              'frequency', 'confide', 'coop', 'authorities', 'coop_reduces']

print('Missing values by model variable (of 212)')
miss = d[model_vars].isna().sum()
for v in model_vars:
    print('  %-14s %3d  (%.1f%%)' % (v, miss[v], miss[v] / len(d) * 100))
print('\nComplete on all model variables and the outcome: %d' %
      d[['ccas_total'] + model_vars].dropna().shape[0])
```

```
Missing values by model variable (of 212)
  age              0  (0.0%)
  income           0  (0.0%)
  education        7  (3.3%)
  female           0  (0.0%)
  severity        11  (5.2%)
  n_losses         0  (0.0%)
  area_type        0  (0.0%)
  frequency        0  (0.0%)
  confide          0  (0.0%)
  coop             0  (0.0%)
  authorities      0  (0.0%)
  coop_reduces     0  (0.0%)
```

```
Complete on all model variables and the outcome: 196
```

Flood severity is the only variable with appreciable missingness (11 of 212, 5.2 per cent). Those respondents had not experienced a flood or declined to rate it. The manuscript reports this figure rather than a threshold and uses complete-case analysis throughout.

1.3 3. Reliability and descriptive statistics

Manuscript: alpha = 0.88 for the 13-item scale, 0.82 cognitive-emotional, 0.85 functional. Total 3.01 (0.72), cognitive-emotional 2.88 (0.74), functional 3.21 (0.89). High band 48 of 212 (22.6%), moderate 124 of 212 (58.5%), low 40 of 212 (18.9%).

[4]:
```
def cronbach_alpha(frame):
    x = frame.dropna()
    k = x.shape[1]
    return k / (k - 1) * (1 - x.var(ddof=1).sum() / x.sum(axis=1).var(ddof=1))


print("Cronbach's alpha")
print('  Full CCAS, 13 items      %.2f' % cronbach_alpha(items))
print('  Cognitive-emotional, 8   %.2f' % cronbach_alpha(items[cog_emo]))
print('  Functional, 5            %.2f' % cronbach_alpha(items[functional]))

print('\nScale scores, mean (SD)')
for name, s in [('Total CCAS', d['ccas_total']), ('Cognitive-emotional',␣
 ↪d['ccas_ce']),
                ('Functional', d['ccas_fn'])]:
    print('  %-20s %.2f (%.2f)' % (name, s.mean(), s.std(ddof=1)))
t_sub = stats.ttest_rel(d['ccas_fn'], d['ccas_ce'])
print('  Functional minus cognitive-emotional, paired t(%d) = %.2f, p = %.3g'
      % (len(d) - 1, t_sub.statistic, t_sub.pvalue))

print('\nAnxiety bands')
n = len(d)
for lab in ['High', 'Moderate', 'Low']:
    c = int((d['band'] == lab).sum())
    print('  %-9s %3d of %d  (%.1f%%)' % (lab, c, n, c / n * 100))

print('\nItem means, ordered')
print(items.mean().sort_values(ascending=False).round(2).to_string())

# corrected item-total correlation: each item against the sum of the other␣
 ↪twelve,
# so the item is not correlated with itself
_tot = items.sum(axis=1)
item_total = pd.Series({c: items[c].corr(_tot - items[c]) for c in items.
 ↪columns})
```

```
print('\nCorrected item-total correlations, ordered')
print(item_total.sort_values(ascending=False).round(2).to_string())
weak = item_total[item_total < 0.30]
print('  Items below 0.30: %d of %d - %s'
      % (len(weak), len(item_total), ', '.join(weak.index)))

print('\nAlpha if the weak items were dropped')
_base = cronbach_alpha(items)
for c in weak.index:
    a = cronbach_alpha(items.drop(columns=[c]))
    print('  without %-28s %.4f  (change %+.4f)' % (c, a, a - _base))
```

```
Cronbach's alpha
  Full CCAS, 13 items      0.88
  Cognitive-emotional, 8   0.82
  Functional, 5            0.85

Scale scores, mean (SD)
  Total CCAS           3.01 (0.72)
  Cognitive-emotional  2.88 (0.74)
  Functional           3.21 (0.89)
  Functional minus cognitive-emotional, paired t(211) = 6.87, p = 7.26e-11

Anxiety bands
  High       48 of 212  (22.6%)
  Moderate  124 of 212  (58.5%)
  Low        40 of 212  (18.9%)

Item means, ordered
CCAS_Q18_WorkStudies          3.50
CCAS_Q17_BalanceDaily         3.43
CCAS_Q8_Concentrate           3.42
CCAS_Q9_Sleep                 3.35
CCAS_Q19_Potential            3.33
CCAS_Q16_FamilyFriends        3.29
CCAS_Q14_ReflectDeeply        3.17
CCAS_Q15_QuestionReact        2.72
CCAS_Q10_Nightmares           2.71
CCAS_Q11_Crying               2.69
CCAS_Q13_PreferAlone          2.55
CCAS_Q20_OthersToldTooMuch    2.50
CCAS_Q12_HandleBetter         2.46

Corrected item-total correlations, ordered
CCAS_Q17_BalanceDaily         0.76
CCAS_Q9_Sleep                 0.75
CCAS_Q18_WorkStudies          0.69
```

```
CCAS_Q8_Concentrate           0.66
CCAS_Q10_Nightmares           0.65
CCAS_Q19_Potential            0.64
CCAS_Q11_Crying               0.62
CCAS_Q14_ReflectDeeply        0.61
CCAS_Q16_FamilyFriends        0.59
CCAS_Q15_QuestionReact        0.48
CCAS_Q20_OthersToldTooMuch    0.46
CCAS_Q13_PreferAlone          0.28
CCAS_Q12_HandleBetter         0.22
  Items below 0.30: 2 of 13 - CCAS_Q12_HandleBetter, CCAS_Q13_PreferAlone

Alpha if the weak items were dropped
  without CCAS_Q12_HandleBetter        0.8925  (change +0.0078)
  without CCAS_Q13_PreferAlone         0.8894  (change +0.0047)
```

1.4 4. Factor structure

Manuscript: KMO = 0.88, Bartlett chi-square(78) = 1425.0, two factors explaining 57.4 per cent.

[5]:
```
corr = np.corrcoef(items.values, rowvar=False)
p = corr.shape[0]
n_efa = len(items)

# Bartlett's test of sphericity
chi_sq = -(n_efa - 1 - (2 * p + 5) / 6) * np.log(np.linalg.det(corr))
df_bart = int(p * (p - 1) / 2)
print("Bartlett chi-square(%d) = %.1f, p = %.3g" % (df_bart, chi_sq, stats.chi2.
 ↪sf(chi_sq, df_bart)))

# Kaiser-Meyer-Olkin from the anti-image (partial) correlations
inv = np.linalg.inv(corr)
scale = np.sqrt(np.diag(inv))
partial = -inv / np.outer(scale, scale)
np.fill_diagonal(partial, 0)
off = ~np.eye(p, dtype=bool)
kmo = (corr[off] ** 2).sum() / ((corr[off] ** 2).sum() + (partial[off] ** 2).
 ↪sum())
print('KMO overall = %.2f' % kmo)

eig = np.sort(np.linalg.eigvalsh(corr))[::-1]
print('\nEigenvalues above 1: %d' % (eig > 1).sum())
print('First four eigenvalues:', np.round(eig[:4], 2))
print('Variance explained by two factors: %.1f%%' % (eig[:2].sum() / p * 100))
print('Cumulative for two factors:', np.round(np.cumsum(eig[:2] / p * 100), 1))
```

```
Bartlett chi-square(78) = 1425.0, p = 1.92e-246
KMO overall = 0.88
```

```
Eigenvalues above 1: 3
First four eigenvalues: [5.72 1.74 1.19 0.85]
Variance explained by two factors: 57.4%
Cumulative for two factors: [44.  57.4]
```

Two components were retained on the basis of the scree plot and the theoretical two-factor structure of the original scale, not the Kaiser criterion, which would have retained three.

[6]:
```
def varimax(loadings, tol=1e-6, max_iter=200):
    L = loadings.copy()
    n_row, n_col = L.shape
    R = np.eye(n_col)
    last = 0.0
    for _ in range(max_iter):
        Lr = L @ R
        u, s, vt = np.linalg.svd(L.T @ (Lr ** 3 - Lr @ np.diag((Lr ** 2).
 ↪sum(0)) / n_row))
        R = u @ vt
        if abs(s.sum() - last) < tol:
            break
        last = s.sum()
    return L @ R


vals, vecs = np.linalg.eigh(corr)
keep = np.argsort(vals)[::-1][:2]
rotated = varimax(vecs[:, keep] * np.sqrt(vals[keep]))

# sign convention: make each factor load positively on the items that define it
rotated = rotated * np.sign(rotated.sum(axis=0))

load = pd.DataFrame(rotated, index=ccas_items, columns=['Factor 1', 'Factor␣
 ↪2']).round(2)
load['subscale'] = ['Cognitive-emotional'] * len(cog_emo) + ['Functional'] *␣
 ↪len(functional)
print(load.to_string())

comm = pd.Series((rotated ** 2).sum(axis=1), index=ccas_items)
print('\nCommunalities, lowest first')
print(comm.sort_values().round(2).to_string())

# an item is treated as cross-loading if it exceeds 0.35 on both rotated factors
cross = load.index[(load['Factor 1'].abs() > 0.35) & (load['Factor 2'].abs() >␣
 ↪0.35)]
print('\nItems loading above 0.35 on both factors: %d - %s' % (len(cross), ', '.
 ↪join(cross)))
```

```
                            Factor 1  Factor 2            subscale
CCAS_Q8_Concentrate             0.78      0.16  Cognitive-emotional
CCAS_Q9_Sleep                   0.74      0.36  Cognitive-emotional
CCAS_Q10_Nightmares             0.62      0.39  Cognitive-emotional
CCAS_Q11_Crying                 0.47      0.57  Cognitive-emotional
CCAS_Q12_HandleBetter          -0.01      0.53  Cognitive-emotional
CCAS_Q13_PreferAlone           -0.07      0.76  Cognitive-emotional
CCAS_Q14_ReflectDeeply          0.40      0.63  Cognitive-emotional
CCAS_Q15_QuestionReact          0.21      0.71  Cognitive-emotional
CCAS_Q16_FamilyFriends          0.75      0.06           Functional
CCAS_Q17_BalanceDaily           0.84      0.20           Functional
CCAS_Q18_WorkStudies            0.86      0.06           Functional
CCAS_Q19_Potential              0.85      0.01           Functional
CCAS_Q20_OthersToldTooMuch      0.35      0.44           Functional

Communalities, lowest first
CCAS_Q12_HandleBetter         0.28
CCAS_Q20_OthersToldTooMuch    0.32
CCAS_Q10_Nightmares           0.54
CCAS_Q11_Crying               0.54
CCAS_Q15_QuestionReact        0.55
CCAS_Q14_ReflectDeeply        0.56
CCAS_Q16_FamilyFriends        0.57
CCAS_Q13_PreferAlone          0.58
CCAS_Q8_Concentrate           0.63
CCAS_Q9_Sleep                 0.67
CCAS_Q19_Potential            0.72
CCAS_Q18_WorkStudies          0.73
CCAS_Q17_BalanceDaily         0.75

Items loading above 0.35 on both factors: 4 - CCAS_Q9_Sleep,
CCAS_Q10_Nightmares, CCAS_Q11_Crying, CCAS_Q14_ReflectDeeply
```

1.5 5. Table 1. Sample characteristics

[7]:
```
def cat_table(series, order=None):
    vc = series.value_counts(dropna=False)
    if order is not None:
        vc = vc.reindex(order).fillna(0).astype(int)
    return pd.DataFrame({'n': vc, '%': (vc / len(series) * 100).round(1)})


print('Age: mean %.1f, SD %.1f, range %d to %d'
      % (d['age'].mean(), d['age'].std(ddof=1), d['age'].min(), d['age'].max()))
print('Monthly household income (BDT): median %.0f, IQR %.0f to %.0f'
      % (d['income'].median(), d['income'].quantile(.25), d['income'].quantile(.
 ↪75)))
```

```
print('\nGender');          print(cat_table(raw['Q40_Gender']).to_string())
print('\nEducation');       print(cat_table(raw['Q36_Education'],␣
  ↪list(edu_map)).to_string())
print('\nMarital status');  print(cat_table(raw['Q37_Marital']).to_string())
print('\nOccupation');      print(cat_table(raw['Q38_Occupation']).to_string())
print('\nResidence type');  print(cat_table(raw['Q2_ResidenceType']).
  ↪to_string())
print('\nYears in the city'); print(cat_table(raw['Q1_YearsInCity'],␣
  ↪list(tenure_map)).to_string())
```

```
Age: mean 39.4, SD 13.4, range 18 to 76
Monthly household income (BDT): median 15000, IQR 8000 to 25000

Gender
              n     %
Q40_Gender
Male        107  50.5
Female      105  49.5

Education
                               n     %
Q36_Education
No formal education           25  11.8
Primary school                61  28.8
Secondary/SSC/O-Level         56  26.4
Higher Secondary/HSC/A-Level  32  15.1
Bachelor's degree             28  13.2
Master's degree or higher      3   1.4
Other                          7   3.3

Marital status
                         n     %
Q37_Marital
Married                177  83.5
Single/Never married    28  13.2
Widowed                  5   2.4
Divorced                 1   0.5
Prefer not to say        1   0.5

Occupation
                                        n     %
Q38_Occupation
Housewife                              85  40.1
Business Owner                         43  20.3
Employed (Private Sector)              20   9.4
Student                                16   7.5
```

```
Farmer/Laborer/Rikshaw Puller/Auto driver  13   6.1
Other                                      11   5.2
Employed (Public Sector)                    8   3.8
Unemployed                                  7   3.3
Abroad                                      3   1.4
Retired                                     3   1.4
School Teacher                              2   0.9
Madrasha Teacher                            1   0.5

Residence type
                                           n     %
Q2_ResidenceType
Own house                                134  63.2
Rented house                              70  33.0
Living with family (Joint Family system)   4   1.9
Institutional: Hostels, dormitories        2   0.9
Government Housing                         1   0.5
Other                                      1   0.5

Years in the city
                       n     %
Q1_YearsInCity
1-5 years             20   9.4
6-10 years            19   9.0
More than 10 years   173  81.6
```

1.6 6. Flood exposure and reported losses

Manuscript: 203 of 212 (95.8%) had experienced flooding, 164 of 212 (77.4%) said their area floods every year, 134 of 212 (63.2%) live in a floodplain or low-lying area. Of the 201 who rated severity, 88 (43.8%) said very severe and 99 (49.3%) moderate. The loss percentages use the 201 respondents who reported at least one loss as the denominator, which is stated in the caption of Fig 1.

[8]:
```
def share(mask, denom=None):
    denom = len(mask) if denom is None else denom
    return '%d of %d (%.1f%%)' % (mask.sum(), denom, mask.sum() / denom * 100)


print('Ever experienced a flood      :', share(raw['Q5_ExperiencedFlood'] ==␣
 ↪'Yes'))
print('Area floods annually          :', share(raw['Q4_FloodFrequency'] ==␣
 ↪'Annually'))
print('Floodplain or low-lying area  :', share(raw['Q3_AreaType'] ==␣
 ↪'Floodplain/Low-lying area'))

print('\nPerceived severity of the worst flood experienced')
print(cat_table(raw['Q6_FloodSeverity'].dropna(), list(severity_map)).
 ↪to_string())
```

```
reported = losses[losses.sum(axis=1) > 0]
n_loss = len(reported)
print('\nRespondents reporting at least one loss: %d' % n_loss)
loss_labels = {
    'Q7_Loss_HouseProperty': 'Damage to house or property',
    'Q7_Loss_Utilities': 'Loss of utilities',
    'Q7_Loss_Infrastructure': 'Damage to local infrastructure',
    'Q7_Loss_IncomeLoss': 'Loss of income',
    'Q7_Loss_PsychStress': 'Psychological stress',
    'Q7_Loss_Sanitation': 'Sanitation problems',
    'Q7_Loss_HealthRisks': 'Health risks',
    'Q7_Loss_SafetyRisks': 'Safety risks',
    'Q7_Loss_Education': 'Disruption to education',
    'Q7_Loss_CropLoss': 'Crop loss',
    'Q7_Loss_Livestock': 'Livestock loss',
    'Q7_Loss_Vehicles': 'Damage to vehicles',
    'Q7_Loss_Debt': 'Debt',
    'Q7_Loss_SocialInsecurity': 'Social insecurity',
}
loss_tab = (pd.DataFrame({'n': reported.sum()})
            .assign(**{'% of 201': lambda t: (t['n'] / n_loss * 100).round(1)})
            .rename(index=loss_labels)
            .sort_values('n', ascending=False))
print(loss_tab.to_string())

print('\nNumber of loss types per respondent: mean %.2f, median %.0f, max %d'
      % (d['n_losses'].mean(), d['n_losses'].median(), d['n_losses'].max()))
```

```
Ever experienced a flood      : 203 of 212 (95.8%)
Area floods annually          : 164 of 212 (77.4%)
Floodplain or low-lying area  : 134 of 212 (63.2%)

Perceived severity of the worst flood experienced
                                                        n     %
Q6_FloodSeverity
Mild (Minor inconvenience)                             14   7.0
Moderate (Damaged homes or roads, but not destructive) 99  49.3
Very Severe (Destroyed homes and infrastructure)       88  43.8

Respondents reporting at least one loss: 201
                                n  % of 201
Damage to house or property   155      77.1
Loss of utilities             151      75.1
Loss of income                130      64.7
Damage to local infrastructure 130     64.7
Health risks                  101      50.2
```

```
Psychological stress              83      41.3
Safety risks                      74      36.8
Sanitation problems               70      34.8
Social insecurity                 54      26.9
Disruption to education           39      19.4
Damage to vehicles                32      15.9
Debt                              26      12.9
Livestock loss                    21      10.4
Crop loss                         14       7.0

Number of loss types per respondent: mean 5.09, median 5, max 12
```

1.7 7. Table S1. Bivariate associations with total CCAS

Thirteen tests: ten Spearman correlations, two one-way ANOVAs for the unordered grouping variables, and a Welch t-test for gender. Benjamini-Hochberg is applied across all thirteen together, which is the reason community cooperation and length of residence fall just short of significance after correction.

[9]:
```
def bh_fdr(pvals):
    p = np.asarray(pvals, float)
    m = len(p)
    order = np.argsort(p)
    adj = np.empty(m)
    adj[order] = np.minimum.accumulate((p[order] * m / np.arange(1, m + 1))[::
 ↪-1])[::-1]
    return np.clip(adj, 0, 1)


spearman_vars = {'severity': 'Flood severity', 'n_losses': 'Number of losses',
                 'income': 'Monthly income', 'frequency': 'Flood frequency',
                 'education': 'Education', 'coop': 'Community cooperation␣
 ↪frequency',
                 'tenure': 'Length of residence', 'coop_reduces': 'Cooperation␣
 ↪reduces impact',
                 'authorities': 'Authorities respond', 'age': 'Age'}

rows = []
for v, lab in spearman_vars.items():
    pair = d[['ccas_total', v]].dropna()
    rho, pv = stats.spearmanr(pair['ccas_total'], pair[v])
    rows.append({'variable': lab, 'test': 'Spearman', 'n': len(pair),
                 'statistic': round(rho, 3), 'p_raw': pv})

for col, lab in [('Q3_AreaType', 'Area flood risk'), ('Q25_TalkTo', 'Person␣
 ↪confided in')]:
    g = pd.DataFrame({'y': d['ccas_total'], 'g': raw[col]}).dropna()
    groups = [x['y'].values for _, x in g.groupby('g')]
```

```
    F, pv = stats.f_oneway(*groups)
    rows.append({'variable': lab, 'test': 'ANOVA F(%d,%d)' % (len(groups) - 1,␣
 ↪len(g) - len(groups)),
                 'n': len(g), 'statistic': round(F, 2), 'p_raw': pv})

g = pd.DataFrame({'y': d['ccas_total'], 'g': raw['Q40_Gender']}).dropna()
t, pv = stats.ttest_ind(g.loc[g.g == 'Female', 'y'], g.loc[g.g == 'Male', 'y'],␣
 ↪equal_var=False)
rows.append({'variable': 'Gender', 'test': 'Welch t', 'n': len(g),
             'statistic': round(t, 2), 'p_raw': pv})

tab2 = pd.DataFrame(rows)
tab2['p_fdr'] = bh_fdr(tab2['p_raw'].values)
tab2 = tab2.sort_values('p_raw').reset_index(drop=True)
tab2['significant'] = np.where(tab2['p_fdr'] < ALPHA, 'yes', '')
print(tab2.assign(p_raw=tab2.p_raw.map('{:.4f}'.format),
                  p_fdr=tab2.p_fdr.map('{:.4f}'.format)).to_string(index=False))

print('\nGroup means behind the two ANOVAs')
for col, lab in [('Q3_AreaType', 'Area flood risk'), ('Q25_TalkTo', 'Person␣
 ↪confided in')]:
    g = pd.DataFrame({'y': d['ccas_total'], 'g': raw[col]}).dropna()
    print('\n' + lab)
    print(g.groupby('g')['y'].agg(n='size', mean='mean', sd=lambda s: s.
 ↪std(ddof=1))
          .round(2).to_string())
```

```
                      variable           test   n  statistic  p_raw  p_fdr
significant
                Flood severity       Spearman 201      0.386 0.0000 0.0000
yes
              Number of losses       Spearman 212      0.341 0.0000 0.0000
yes
               Area flood risk ANOVA F(2,209) 212     14.740 0.0000 0.0000
yes
                Monthly income       Spearman 212     -0.275 0.0000 0.0002
yes
               Flood frequency       Spearman 212      0.256 0.0002 0.0004
yes
            Person confided in ANOVA F(2,209) 212      8.090 0.0004 0.0009
yes
                     Education       Spearman 205     -0.175 0.0120 0.0224
yes
Community cooperation frequency       Spearman 212     -0.144 0.0358 0.0563
           Length of residence       Spearman 212      0.142 0.0390 0.0563
    Cooperation reduces impact       Spearman 212     -0.124 0.0712 0.0926
           Authorities respond       Spearman 212      0.120 0.0806 0.0953
```

```
                          Age       Spearman 212      0.081 0.2397 0.2596
                       Gender        Welch t 212      0.970 0.3325 0.3325

Group means behind the two ANOVAs

Area flood risk
                              n  mean    sd
g
Elevated area                 7  2.20  0.62
Floodplain/Low-lying area   134  3.19  0.65
Moderately flood-prone       71  2.75  0.74

Person confided in

                                 n  mean    sd
g
No one                           5  2.37  1.03
Talk to family members         166  3.11  0.68
Talk to friends or neighbors    41  2.69  0.75
```

1.8 8. Table 2. Hierarchical regression on total CCAS

Block 1 socio-demographics, block 2 adds flood exposure, block 3 adds the community and institutional variables. Everything is z-scored, standard errors are HC3, and all three blocks use the same 196 complete cases.

[10]:
```
block1 = ['age', 'income', 'education', 'female']
block2 = block1 + ['severity', 'n_losses', 'area_type', 'frequency']
block3 = block2 + ['confide', 'coop', 'authorities', 'coop_reduces']

pretty = {'age': 'Age', 'income': 'Monthly income', 'education': 'Education',
          'female': 'Female', 'severity': 'Flood severity', 'n_losses': 'Number␣
 ↪of losses',
          'area_type': 'Area flood risk', 'frequency': 'Flood frequency',
          'confide': 'Confides in friends or neighbours', 'coop': 'Cooperation␣
 ↪frequency',
          'authorities': 'Authorities respond', 'coop_reduces': 'Cooperation␣
 ↪reduces impact'}


def zscore(frame):
    return frame.apply(lambda c: (c - c.mean()) / c.std(ddof=1))


def ols_hc3(outcome, cols, data=d, sample=None):
    """Least squares with HC3 robust standard errors, on standardized variables.
```

```
    `sample` restricts the rows so that competing blocks share one listwise␣
↪sample.
    Returns the coefficient table, R-squared, adjusted R-squared and N.
    """
    frame = pd.concat([data[outcome].rename('y'), data[cols]], axis=1).dropna()
    if sample is not None:
        frame = frame.loc[frame.index.intersection(sample)]
    z = zscore(frame)
    y = z['y'].values
    X = np.column_stack([np.ones(len(z))] + [z[c].values for c in cols])
    n_obs, k = X.shape

    XtX_inv = np.linalg.inv(X.T @ X)
    beta = XtX_inv @ X.T @ y
    resid = y - X @ beta
    lev = np.einsum('ij,jk,ik->i', X, XtX_inv, X)

    # HC3: weight each squared residual by 1/(1-h)^2
    meat = X.T @ np.diag(resid ** 2 / (1 - lev) ** 2) @ X
    cov = XtX_inv @ meat @ XtX_inv
    se = np.sqrt(np.diag(cov))
    tval = beta / se
    dfree = n_obs - k
    pval = 2 * stats.t.sf(np.abs(tval), dfree)
    crit = stats.t.ppf(1 - ALPHA / 2, dfree)

    r2 = 1 - (resid ** 2).sum() / ((y - y.mean()) ** 2).sum()
    adj = 1 - (1 - r2) * (n_obs - 1) / dfree

    table = pd.DataFrame({'beta': beta[1:], 'SE': se[1:],
                          'CI_low': beta[1:] - crit * se[1:],
                          'CI_high': beta[1:] + crit * se[1:],
                          't': tval[1:], 'p': pval[1:]},
                         index=[pretty[c] for c in cols]).round(3)
    # the rounded p column above turns anything under 0.0005 into "0.001" and
    # anything under 0.0005 in the other direction into "0.000", so the column
    # below carries the value as it should be reported: three decimals down to
    # 0.001, and "<0.001" beneath that
    table['p reported'] = ['<0.001' if v < 0.001 else '%.3f' % v for v in␣
↪pval[1:]]
    return table, r2, adj, n_obs, resid, lev, X, z


# the listwise sample for the full model, reused by every block
analytic = d[['ccas_total'] + block3].dropna().index
print('Analytic sample for all three blocks: N = %d' % len(analytic))
```

```
prev_r2, prev_k = 0.0, 0
for i, blk in enumerate([block1, block2, block3], start=1):
    tbl, r2, adj, N, *_ = ols_hc3('ccas_total', blk, sample=analytic)
    k = len(blk)
    if i == 1:
        # no earlier block to compare with, so test the block against the null␣
 ↪model
        F = (r2 / k) / ((1 - r2) / (N - k - 1))
        pF = stats.f.sf(F, k, N - k - 1)
        line = '   F(%d, %d) = %.2f, p = %.3f' % (k, N - k - 1, F, pF)
    else:
        F = ((r2 - prev_r2) / (k - prev_k)) / ((1 - r2) / (N - k - 1))
        pF = stats.f.sf(F, k - prev_k, N - k - 1)
        line = '   change in R2 = %.3f, F(%d, %d) = %.2f, p = %.3g' % (
            r2 - prev_r2, k - prev_k, N - k - 1, F, pF)
    print('\n===== Block %d, N = %d =====' % (i, N))
    print(tbl.to_string())
    print('R2 = %.3f, adjusted R2 = %.3f%s' % (r2, adj, line))
    prev_r2, prev_k = r2, k
```

```
Analytic sample for all three blocks: N = 196

===== Block 1, N = 196 =====
                 beta     SE  CI_low  CI_high      t      p p reported
Age             0.048  0.083  -0.115    0.211  0.583  0.560      0.560
Monthly income -0.039  0.047  -0.133    0.054 -0.832  0.406      0.406
Education      -0.081  0.080  -0.238    0.077 -1.012  0.313      0.313
Female          0.081  0.075  -0.067    0.229  1.074  0.284      0.284
R2 = 0.023, adjusted R2 = 0.003   F(4, 191) = 1.13, p = 0.345

===== Block 2, N = 196 =====
                   beta     SE  CI_low  CI_high      t      p p reported
Age                0.132  0.077  -0.020    0.283  1.714  0.088      0.088
Monthly income    -0.028  0.057  -0.141    0.085 -0.485  0.628      0.628
Education          0.022  0.074  -0.125    0.168  0.292  0.771      0.771
Female             0.081  0.068  -0.054    0.216  1.179  0.240      0.240
Flood severity     0.263  0.072   0.121    0.405  3.660  0.000     <0.001
Number of losses   0.247  0.069   0.111    0.383  3.571  0.000     <0.001
Area flood risk    0.130  0.081  -0.030    0.289  1.607  0.110      0.110
Flood frequency    0.092  0.057  -0.020    0.204  1.619  0.107      0.107
R2 = 0.242, adjusted R2 = 0.210   change in R2 = 0.219, F(4, 187) = 13.54, p =
1.03e-09

===== Block 3, N = 196 =====
                                   beta     SE  CI_low  CI_high      t      p p
reported
Age                               0.116  0.072  -0.026    0.259  1.610  0.109
```

```
0.109
Monthly income                     -0.020  0.059  -0.136    0.097 -0.332  0.740
0.740
Education                           0.051  0.073  -0.093    0.195  0.699  0.486
0.486
Female                              0.023  0.071  -0.117    0.162  0.321  0.749
0.749
Flood severity                      0.261  0.074   0.115    0.408  3.519  0.001
<0.001
Number of losses                    0.203  0.061   0.083    0.322  3.353  0.001
<0.001
Area flood risk                     0.139  0.076  -0.010    0.289  1.837  0.068
0.068
Flood frequency                     0.046  0.060  -0.073    0.164  0.761  0.447
0.447
Confides in friends or neighbours -0.143  0.074  -0.290    0.004 -1.924  0.056
0.056
Cooperation frequency              -0.151  0.088  -0.325    0.023 -1.708  0.089
0.089
Authorities respond                 0.167  0.071   0.027    0.308  2.348  0.020
0.020
Cooperation reduces impact         -0.087  0.079  -0.243    0.069 -1.098  0.274
0.274
R2 = 0.314, adjusted R2 = 0.269   change in R2 = 0.072, F(4, 183) = 4.78, p =
0.00109
```

1.8.1 Table 2 as it is printed in the manuscript

The three blocks are laid side by side so that a reader can see whether a coefficient moves when a later block enters. Only the final model carries a confidence interval and a p-value here; Online Resource 7 has the standard errors and the per-block intervals. Rows are ordered by the size of the final coefficient, largest first.

[11]:
```
ms_label = {'age': 'Age', 'income': 'Income', 'education': 'Education',
            'female': 'Female', 'severity': 'Flood severity',
            'n_losses': 'Number of losses', 'area_type': 'Area flood risk',
            'frequency': 'Flood frequency',
            'confide': 'Confides in friends/neighbours',
            'coop': 'Community cooperation freq.',
            'authorities': 'Perceives authorities effective',
            'coop_reduces': 'Cooperation reduces impact'}

fits = {}
for i, blk in enumerate([block1, block2, block3], start=1):
    tbl, r2, adj, N, *_ = ols_hc3('ccas_total', blk, sample=analytic)
    tbl.index = [ms_label[c] for c in blk]
    fits[i] = (tbl, r2, adj)
```

```
def signed(v):
    """+0.26 style, so that a sign change between blocks is visible at a glance.
    """
    return '%+.2f' % v


b3 = fits[3][0]
order = b3['beta'].abs().sort_values(ascending=False).index

rows = []
for name in order:
    cells = []
    for i in (1, 2):
        t = fits[i][0]
        cells.append(signed(t.loc[name, 'beta']) if name in t.index else␣
    '\u2014')
    lo, hi = b3.loc[name, 'CI_low'], b3.loc[name, 'CI_high']
    cells.append('%s (%s, %s)' % (signed(b3.loc[name, 'beta']), signed(lo),␣
    signed(hi)))
    cells.append(b3.loc[name, 'p reported'])
    rows.append([name] + cells)

rows.append(['R2'] + ['%.2f' % fits[i][1] for i in (1, 2, 3)] + [''])
rows.append(['Adjusted R2'] + ['%.2f' % fits[i][2] for i in (1, 2, 3)] + [''])
rows.append(['Change in R2', '\u2014',
             '%.2f' % (fits[2][1] - fits[1][1]),
             '%.2f' % (fits[3][1] - fits[2][1]), ''])

table2 = pd.DataFrame(rows, columns=['Predictor', 'Block 1', 'Block 2',
                                     'Block 3 beta (95% CI)', 'p'])
print(table2.to_string(index=False))
```

Predictor	Block 1	Block 2	Block 3 beta (95% CI)	p
Flood severity	-	+0.26	+0.26 (+0.12, +0.41)	<0.001
Number of losses	-	+0.25	+0.20 (+0.08, +0.32)	<0.001
Perceives authorities effective	-	-	+0.17 (+0.03, +0.31)	0.020
Community cooperation freq.	-	-	−0.15 (−0.33, +0.02)	0.089
Confides in friends/neighbours	-	-	−0.14 (−0.29, +0.00)	0.056
Area flood risk	-	+0.13	+0.14 (−0.01, +0.29)	0.068
Age	+0.05	+0.13	+0.12 (−0.03, +0.26)	0.109
Cooperation reduces impact	-	-	−0.09 (−0.24, +0.07)	0.274
Education	−0.08	+0.02	+0.05 (−0.09, +0.20)	0.486
Flood frequency	-	+0.09	+0.05 (−0.07, +0.16)	0.447
Female	+0.08	+0.08	+0.02 (−0.12, +0.16)	0.749
Income	−0.04	−0.03	−0.02 (−0.14, +0.10)	0.740
R2	0.02	0.24	0.31	

Adjusted R2	0.00	0.21	0.27
Change in R2	-	0.22	0.07

1.8.2 Regression diagnostics

Manuscript: all VIF at or below 1.86, studentized Breusch-Pagan p = 0.45, maximum Cook's distance 0.05.

[12]:
```
tbl3, r2_3, adj_3, N3, resid, lev, X, z = ols_hc3('ccas_total', block3,␣
 ↪sample=analytic)

# variance inflation from the predictor correlation matrix
Rx = np.corrcoef(z[block3].values, rowvar=False)
vif = pd.Series(np.diag(np.linalg.inv(Rx)), index=[pretty[c] for c in block3]).
 ↪round(2)
print('Variance inflation factors')
print(vif.sort_values(ascending=False).to_string())
print('Maximum VIF = %.2f' % vif.max())

# studentized (Koenker) Breusch-Pagan: regress the scaled squared residuals on␣
 ↪the design
scaled = resid ** 2 / (resid ** 2).mean()
fitted = X @ np.linalg.lstsq(X, scaled, rcond=None)[0]
r2_aux = 1 - ((scaled - fitted) ** 2).sum() / ((scaled - scaled.mean()) ** 2).
 ↪sum()
LM = len(scaled) * r2_aux
print('\nBreusch-Pagan (studentized) LM = %.2f, df = %d, p = %.3f'
      % (LM, X.shape[1] - 1, stats.chi2.sf(LM, X.shape[1] - 1)))

k = X.shape[1]
s2 = (resid ** 2).sum() / (len(resid) - k)
cooks = (resid ** 2 / (s2 * k)) * (lev / (1 - lev) ** 2)
print("Maximum Cook's distance = %.3f (threshold 4/n = %.3f)" % (cooks.max(), 4␣
 ↪/ len(resid)))
print('Observations above the threshold: %d' % (cooks > 4 / len(resid)).sum())
```

```
Variance inflation factors
Cooperation reduces impact           1.86
Cooperation frequency                1.77
Area flood risk                      1.47
Flood severity                       1.44
Education                            1.38
Flood frequency                      1.32
Age                                  1.24
Confides in friends or neighbours    1.24
Female                               1.21
Authorities respond                  1.12
Number of losses                     1.09
```

```
Monthly income                          1.07
Maximum VIF = 1.86

Breusch-Pagan (studentized) LM = 12.00, df = 12, p = 0.446
Maximum Cook's distance = 0.052 (threshold 4/n = 0.020)
Observations above the threshold: 13
```

1.9 9. Table 3. Subscale-specific models

The block 3 specification refitted on each subscale, on the same 196 respondents.

[13]:
```
for label, col in [('Cognitive-emotional impairment', 'ccas_ce'),
                   ('Functional impairment', 'ccas_fn')]:
    tbl, r2, adj, N, *_ = ols_hc3(col, block3, sample=analytic)
    print('===== %s, N = %d =====' % (label, N))
    print(tbl.to_string())
    print('R2 = %.3f, adjusted R2 = %.3f\n' % (r2, adj))
```

```
===== Cognitive-emotional impairment, N = 196 =====
                                    beta     SE  CI_low  CI_high      t      p p
reported
Age                                0.111  0.076  -0.039    0.260  1.463  0.145
0.145
Monthly income                    -0.002  0.056  -0.113    0.108 -0.042  0.967
0.967
Education                          0.041  0.080  -0.117    0.200  0.511  0.610
0.610
Female                             0.113  0.072  -0.029    0.255  1.572  0.118
0.118
Flood severity                     0.271  0.079   0.116    0.427  3.439  0.001
<0.001
Number of losses                   0.114  0.060  -0.004    0.232  1.908  0.058
0.058
Area flood risk                    0.182  0.080   0.025    0.339  2.285  0.023
0.023
Flood frequency                    0.020  0.064  -0.107    0.147  0.318  0.751
0.751
Confides in friends or neighbours -0.122  0.072  -0.264    0.019 -1.705  0.090
0.090
Cooperation frequency             -0.106  0.091  -0.286    0.075 -1.157  0.249
0.249
Authorities respond                0.119  0.073  -0.025    0.263  1.626  0.106
0.106
Cooperation reduces impact        -0.078  0.081  -0.237    0.081 -0.972  0.332
0.332
R2 = 0.282, adjusted R2 = 0.235


===== Functional impairment, N = 196 =====
```

```
                                    beta     SE  CI_low  CI_high      t      p p
reported
Age                                0.098  0.072  -0.045    0.240  1.350  0.179
0.179
Monthly income                    -0.038  0.056  -0.149    0.073 -0.678  0.499
0.499
Education                          0.052  0.068  -0.082    0.187  0.770  0.442
0.442
Female                            -0.103  0.072  -0.246    0.039 -1.431  0.154
0.154
Flood severity                     0.189  0.082   0.028    0.350  2.312  0.022
0.022
Number of losses                   0.275  0.067   0.142    0.408  4.086  0.000
<0.001
Area flood risk                    0.051  0.077  -0.101    0.202  0.661  0.510
0.510
Flood frequency                    0.069  0.061  -0.051    0.189  1.137  0.257
0.257
Confides in friends or neighbours -0.139  0.078  -0.292    0.015 -1.784  0.076
0.076
Cooperation frequency             -0.177  0.087  -0.348   -0.006 -2.045  0.042
0.042
Authorities respond                0.194  0.067   0.061    0.326  2.888  0.004
0.004
Cooperation reduces impact        -0.079  0.079  -0.236    0.078 -0.994  0.322
0.322
R2 = 0.284, adjusted R2 = 0.237
```

1.10 10. Ranked priorities

Each respondent placed the same four options in order from 1 to 4, so the four columns say what went in each position. Below they are inverted into a position per option, which is the form Fig. 2 needs.

[14]:
```
rank_cols = ['Q33_Rank1', 'Q33_Rank2', 'Q33_Rank3', 'Q33_Rank4']
ranks = raw[rank_cols]
options = sorted(pd.unique(ranks.values.ravel()))
print('Respondents ranking all four options: %d of %d' % (ranks.notna().
 ↪all(axis=1).sum(), len(raw)))

position = pd.DataFrame({opt: ranks.eq(opt).values.argmax(axis=1) + 1 for opt␣
 ↪in options},
                        index=raw.index)

tab5 = pd.DataFrame({
    'mean_rank': position.mean().round(2),
    'ranked_first_n': (position == 1).sum(),
```

```
    'ranked_first_%': ((position == 1).mean() * 100).round(1),
    'top_two_n': (position <= 2).sum(),
    'top_two_%': ((position <= 2).mean() * 100).round(1),
    'ranked_last_n': (position == 4).sum(),
    'ranked_last_%': ((position == 4).mean() * 100).round(1),
}).sort_values('mean_rank')
print()
print(tab5.to_string())
```

```
Respondents ranking all four options: 212 of 212


                                        mean_rank  ranked_first_n
ranked_first_%  top_two_n  top_two_%  ranked_last_n  ranked_last_%
Better urban planning and infrastructure       1.99              86
40.6        157       74.1             28           13.2
Government disaster management                 2.18              63
29.7        143       67.5             33           15.6
Strong community cooperation                   2.80              19
9.0         65       30.7             41           19.3
Social support from family and friends         3.03              44
20.8         59       27.8            110           51.9
```

1.11 11. Open-ended suggestions

The free-text answers to Q34 were coded into themes by keyword and each answer can fall into more than one theme. The keyword lists below are the ones used for the manuscript. They allow for the spellings that actually occur in the responses: "dragging" and "dredgings" for dredging, "khonon" for excavation, and "water extraction" for drainage, which several respondents used for the same idea.

[15]:
```
sugg = raw['Q34_Suggestions'].dropna().astype(str).str.strip()
sugg = sugg[sugg != '']
n_sugg = len(sugg)
print('Respondents giving a suggestion: %d of %d' % (n_sugg, len(raw)))

low = sugg.str.lower()
themes = {
    'Improve drainage': r'drain|water extraction|watrer extraction',
    'Dredge or excavate rivers and canals':␣
 ↪r'dredg|dragg|excavat|khonon|digging',
    'Embankments, guard walls, sluice gates': r'embank|guard␣
 ↪wall|barrage|switch gate|badh|bridge',
    'Government support, relief or funding':␣
 ↪r'govern|govt|sorker|sarker|srkr|relief|relif|fund|aid|anud',
    'Clean water, medicine or shelter': r'bissudo|bisuddo|bissodo|pure␣
 ↪water|usud|usod|asroy|shelter|cikitsa',
```

```
    'Waste and cleaning':␣
  ↪r'waste|garbage|plastic|polythene|poriskar|poricchonota',
    'Community or awareness measures':␣
  ↪r'communit|aware|educat|train|shikkha|rescue',
}
for name, pat in themes.items():
    hit = low.str.contains(pat, regex=True)
    print('  %-32s %3d of %d  (%.1f%%)' % (name, hit.sum(), n_sugg, hit.mean()␣
  ↪* 100))

hits = pd.DataFrame({name: low.str.contains(pat, regex=True) for name, pat in␣
  ↪themes.items()})

# the three water and infrastructure themes taken together, reported in the␣
  ↪Discussion
water = hits[['Improve drainage', 'Dredge or excavate rivers and canals',
              'Embankments, guard walls, sluice gates']].any(axis=1)
print('\n  %-32s %3d of %d  (%.1f%%)'
      % ('Any water or infrastructure theme', water.sum(), n_sugg, water.mean()␣
  ↪* 100))

none = ~hits.any(axis=1)
print('  %-32s %3d of %d  (%.1f%%)'
      % ('Matching none of the themes', none.sum(), n_sugg, none.mean() * 100))
print('  Themes are not mutually exclusive, so the counts above sum to more␣
  ↪than %d.' % n_sugg)

print('\nMost frequent verbatim answers')
print(low.value_counts().head(10).to_string())
```

```
Respondents giving a suggestion: 211 of 212
  Improve drainage                  86 of 211  (40.8%)
  Dredge or excavate rivers and canals  51 of 211  (24.2%)
  Embankments, guard walls, sluice gates  31 of 211  (14.7%)
  Government support, relief or funding  40 of 211  (19.0%)
  Clean water, medicine or shelter  11 of 211  (5.2%)
  Waste and cleaning                 6 of 211  (2.8%)
  Community or awareness measures    4 of 211  (1.9%)

  Any water or infrastructure theme 143 of 211  (67.8%)
  Matching none of the themes       19 of 211  (9.0%)
  Themes are not mutually exclusive, so the counts above sum to more than 211.

Most frequent verbatim answers
Q34_Suggestions
improvement of drainage system          27
construction of embankments             16
```

```
river dredging                          12
river excavation                         9
improve drainage system                  7
government initiative                    7
no comment                               6
improvement of water drainage system     4
government support                       4
development of drainage system           3
```

1.12 12. Table S2. Neighbourhood variation

Manuscript: $F(12, 199) = 7.96$, ICC = 0.30, permutation $p < 0.001$.

[16]:
```
nbh = pd.DataFrame({'area': d['area'], 'ccas': d['ccas_total'],
                    'severity': d['severity']}).dropna(subset=['area', 'ccas'])
HIGH_TABLE6 = 3.5
g = nbh.groupby('area')['ccas']

tab6 = pd.DataFrame({
    'n': g.size(),
    'mean': g.mean(),
    'sd': g.std(ddof=1),
    'n_high': nbh.assign(hi=nbh['ccas'] >= HIGH_TABLE6).groupby('area')['hi'].
 ↪sum(),
    '%_high': nbh.assign(hi=nbh['ccas'] >= HIGH_TABLE6).groupby('area')['hi'].
 ↪mean() * 100,
    'mean_severity': nbh.groupby('area')['severity'].mean(),
}).sort_values('mean', ascending=False).round(2)
print(tab6.to_string())

groups = [x.values for _, x in g]
k = len(groups)
N = len(nbh)
grand = nbh['ccas'].mean()
ss_between = sum(len(x) * (x.mean() - grand) ** 2 for x in groups)
ss_within = sum(((x - x.mean()) ** 2).sum() for x in groups)
ms_between = ss_between / (k - 1)
ms_within = ss_within / (N - k)
F_obs = ms_between / ms_within
n0 = (N - sum(len(x) ** 2 for x in groups) / N) / (k - 1)
icc = (ms_between - ms_within) / (ms_between + (n0 - 1) * ms_within)

print('\nOne-way ANOVA  F(%d, %d) = %.2f, p = %.3g' % (k - 1, N - k, F_obs,␣
 ↪stats.f.sf(F_obs, k - 1, N - k)))
print('MS between = %.3f, MS within = %.3f, average cluster size = %.2f' %␣
 ↪(ms_between, ms_within, n0))
print('ICC(1) = %.3f' % icc)
```

```
# permutation test, because the group sizes are small and unequal
rng = np.random.default_rng(SEED)
labels = nbh['area'].values
values = nbh['ccas'].values
B = 10000
count = 0
for _ in range(B):
    shuffled = rng.permutation(values)
    s = pd.Series(shuffled).groupby(labels)
    between = (s.size() * (s.mean() - shuffled.mean()) ** 2).sum()
    within = ((shuffled - shuffled.mean()) ** 2).sum() - between
    if (between / (k - 1)) / (within / (N - k)) >= F_obs:
        count += 1
print('Permutation p = %.4f  (%d draws, seed %d)' % ((count + 1) / (B + 1), B,␣
 ↪SEED))
```

area	n	mean	sd	n_high	%_high	mean_severity
Alampur	17	3.59	0.47	11	64.71	2.88
Taltola	22	3.40	0.48	11	50.00	2.00
Burhanabad	28	3.37	0.43	13	46.43	2.89
Kadamtoli	13	3.34	0.58	6	46.15	2.15
Boroikandi	13	3.23	0.69	7	53.85	2.50
Mendibag	20	3.18	0.47	3	15.00	2.40
Uposhahar	10	3.08	0.78	4	40.00	3.00
Topkhana	14	2.69	0.87	3	21.43	2.21
Mirabazar	24	2.69	0.50	0	0.00	2.05
Vataliya	12	2.65	0.85	2	16.67	2.67
Kushighat	11	2.45	0.47	0	0.00	2.00
Kanishail	13	2.45	0.59	1	7.69	1.50
Sheikhghat	15	2.31	0.90	1	6.67	2.13

```
One-way ANOVA  F(12, 199) = 7.96, p = 4.3e-12
MS between = 2.984, MS within = 0.375, average cluster size = 16.16
ICC(1) = 0.301

Permutation p = 0.0001  (10000 draws, seed 42)
```

1.13 13. Figures

Colours are from the Okabe-Ito palette so the figures stay readable in greyscale and for readers with colour vision deficiency. Setting `EXPORT = True` writes each one to TIFF at 300 dpi, sized to the journal's double-column width of 174 mm; otherwise they are drawn inline.

[17]:

```
OKABE = ['#0072B2', '#E69F00', '#009E73', '#D55E00', '#CC79A7', '#56B4E9',␣
 ↪'#F0E442', '#000000']
# figure.autolayout keeps the long y-axis labels inside the canvas. Without it␣
 ↪the
# on-screen window clips them, because savefig.bbox only applies when a file is␣
 ↪written
plt.rcParams.update({'figure.dpi': 110, 'font.size': 10, 'axes.spines.top':␣
 ↪False,
                     'axes.spines.right': False, 'savefig.bbox': 'tight',
                     'figure.autolayout': True})

EXPORT = True                   # set True to write the submission TIFFs
FIG_DIR = 'Figures'
MM = 1 / 25.4                   # millimetres to inches
W2 = 174 * MM                   # double-column width
DPI = 300

def save_fig(fig, name):
    """Write one TIFF at exactly the journal width, RGB on white at 300 dpi.
    savefig.bbox is forced off for the write so that figsize is honoured to the
    pixel; figure.autolayout has already pulled the labels inside the canvas."""
    if not EXPORT:
        return
    os.makedirs(FIG_DIR, exist_ok=True)
    path = os.path.join(FIG_DIR, name + '.tif')
    buf = io.BytesIO()
    with plt.rc_context({'savefig.bbox': None}):
        fig.savefig(buf, dpi=DPI, format='png', facecolor='white')
    buf.seek(0)
    im = Image.open(buf)
    flat = Image.new('RGB', im.size, 'white')
    flat.paste(im, mask=im.split()[-1] if im.mode == 'RGBA' else None)
    flat.save(path, format='TIFF', compression='tiff_lzw', dpi=(DPI, DPI))
    print('%-8s %5.1f x %5.1f mm  %d x %d px  %s' %
          (name, flat.width / DPI / MM, flat.height / DPI / MM,
           flat.width, flat.height, flat.mode))
```

1.13.1 Fig. 1. Distribution of total CCAS scores (a) and mean CCAS by perceived flood severity (b)

The two panels that were Figures 2 and 3 in the earlier draft, combined so that the paper carries one display item instead of two.

[18]:
```
fig, (axA, axB) = plt.subplots(1, 2, figsize=(W2, 3.2))

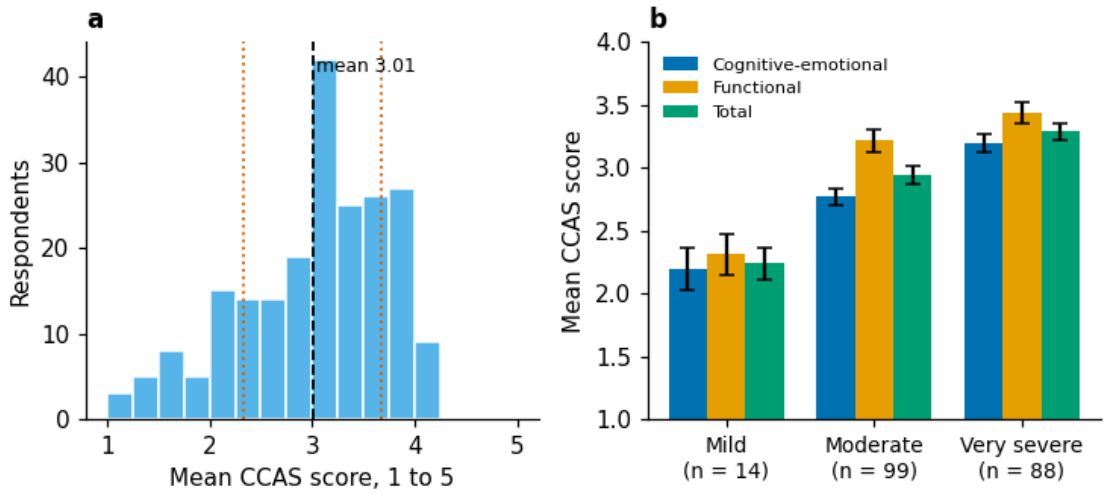
# panel a, distribution of total scores
```

```
axA.hist(d['ccas_total'], bins=np.arange(1, 5.01, 0.25), color=OKABE[5],␣
 ↪edgecolor='white')
axA.axvline(d['ccas_total'].mean(), color='black', ls='--', lw=1.2)
for cut in (LOW_CUT, HIGH_CUT):
    axA.axvline(cut, color=OKABE[3], ls=':', lw=1.2)
axA.text(d['ccas_total'].mean() + 0.03, axA.get_ylim()[1] * 0.92,
         'mean %.2f' % d['ccas_total'].mean(), fontsize=8)
axA.set_xlabel('Mean CCAS score, 1 to 5')
axA.set_ylabel('Respondents')
axA.set_title('a', loc='left', fontweight='bold', fontsize=11)

# panel b, subscale and total means by perceived severity
order = list(severity_map)
short = ['Mild', 'Moderate', 'Very severe']
sub_sev = d.loc[d['severity'].notna()]
width = 0.26
x = np.arange(3)
for j, (col, lab) in enumerate([('ccas_ce', 'Cognitive-emotional'),
                                ('ccas_fn', 'Functional'),
                                ('ccas_total', 'Total')]):
    means = [sub_sev.loc[sub_sev['severity'] == s + 1, col].mean() for s in␣
 ↪range(3)]
    errs = [sub_sev.loc[sub_sev['severity'] == s + 1, col].sem() for s in␣
 ↪range(3)]
    axB.bar(x + (j - 1) * width, means, width, yerr=errs, capsize=3,
            color=OKABE[j], label=lab)
counts = [int((sub_sev['severity'] == s + 1).sum()) for s in range(3)]
axB.set_xticks(x)
axB.set_xticklabels(['%s\n(n = %d)' % (s, c) for s, c in zip(short, counts)],␣
 ↪fontsize=9)
axB.set_ylabel('Mean CCAS score')
axB.set_ylim(1, 4)
axB.legend(frameon=False, fontsize=7.5)
axB.set_title('b', loc='left', fontweight='bold', fontsize=11)

save_fig(fig, 'Fig1')
plt.show()
```

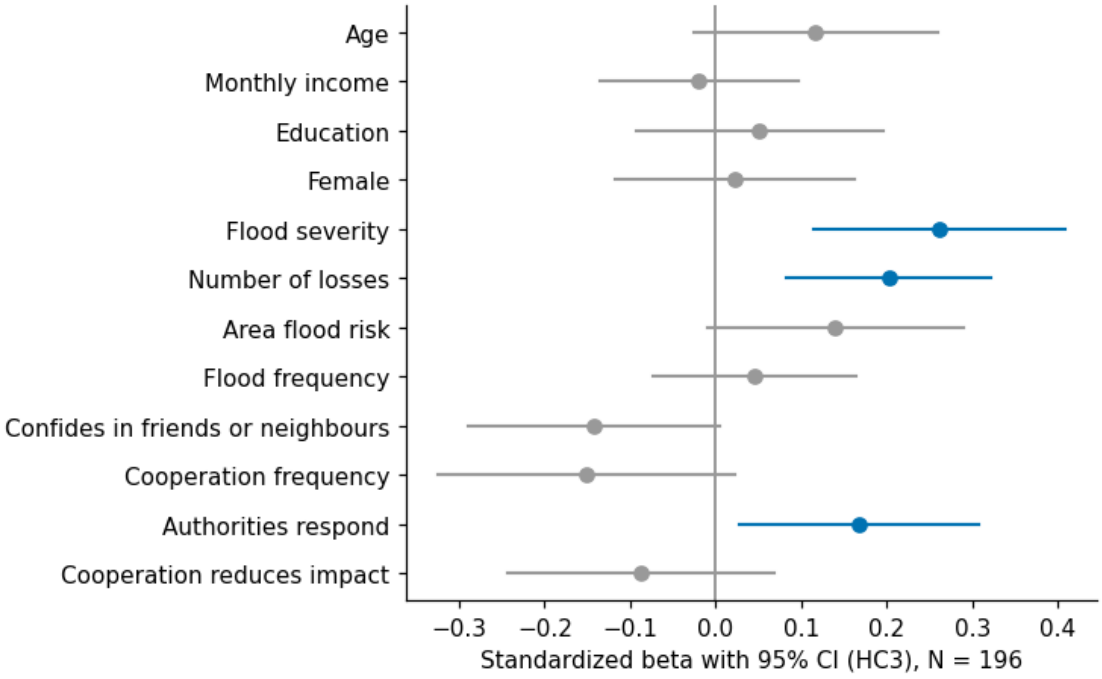
```
Fig1     174.0 x  81.3 mm  2055 x 960 px  RGB
```



1.13.2 Coefficient plot for the fully adjusted model, total CCAS

A working plot only. It is not submitted as a figure, because it shows the same numbers as Table 2 in the manuscript and section A of Online Resource 7. Keeping it here lets a reader check the pattern by eye without spending a display item.

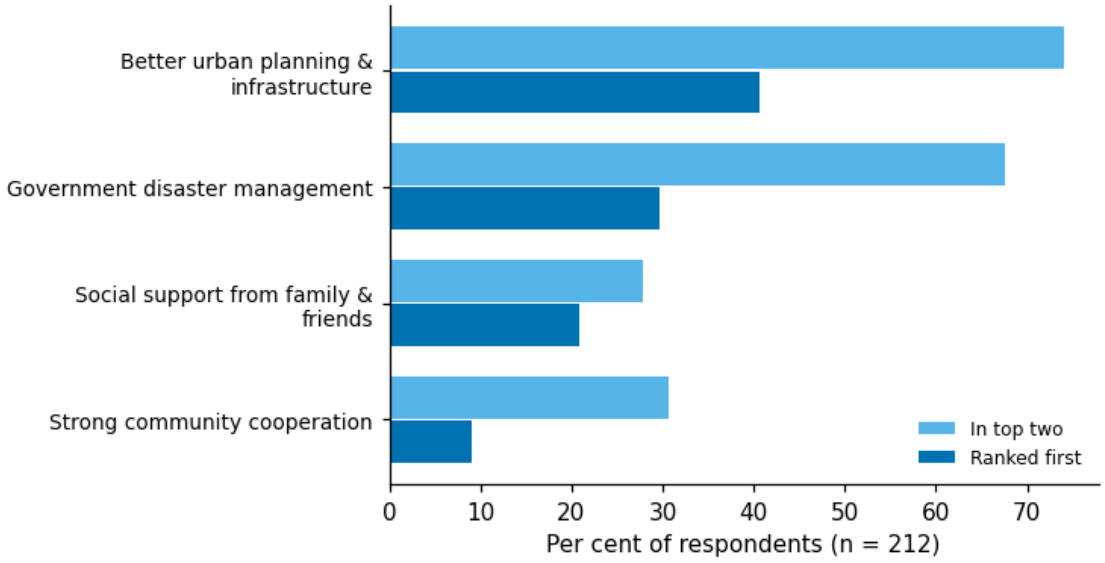
[19]:
```
plot2 = tbl3.iloc[::-1]
fig, ax = plt.subplots(figsize=(W2, 4.3))
ax.axvline(0, color='grey', lw=1)
for i, (name, row) in enumerate(plot2.iterrows()):
    colour = OKABE[0] if row['p'] < ALPHA else '#999999'
    ax.plot([row['CI_low'], row['CI_high']], [i, i], color=colour, lw=1.4)
    ax.plot(row['beta'], i, 'o', color=colour, ms=6)
ax.set_yticks(range(len(plot2)))
ax.set_yticklabels(plot2.index)
ax.set_xlabel('Standardized beta with 95%% CI (HC3), N = %d' % N3)
# not exported; see the note above
plt.show()
```



1.13.3 Fig. 2. Ranked priorities for reducing climate anxiety

[20]:
```
plot3 = tab5.sort_values('ranked_first_%')
fig, ax = plt.subplots(figsize=(W2, 3.6))
y = np.arange(len(plot3))
ax.barh(y + 0.19, plot3['top_two_%'], 0.36, color=OKABE[5], label='In top two')
ax.barh(y - 0.19, plot3['ranked_first_%'], 0.36, color=OKABE[0], label='Ranked␣
 ↪first')
ax.set_yticks(y)
ax.set_yticklabels([t.replace(' and ', ' &\n') for t in plot3.index],␣
 ↪fontsize=9)
ax.set_xlabel('Per cent of respondents (n = %d)' % len(raw))
ax.legend(frameon=False, fontsize=8)
save_fig(fig, 'Fig2')
plt.show()
```

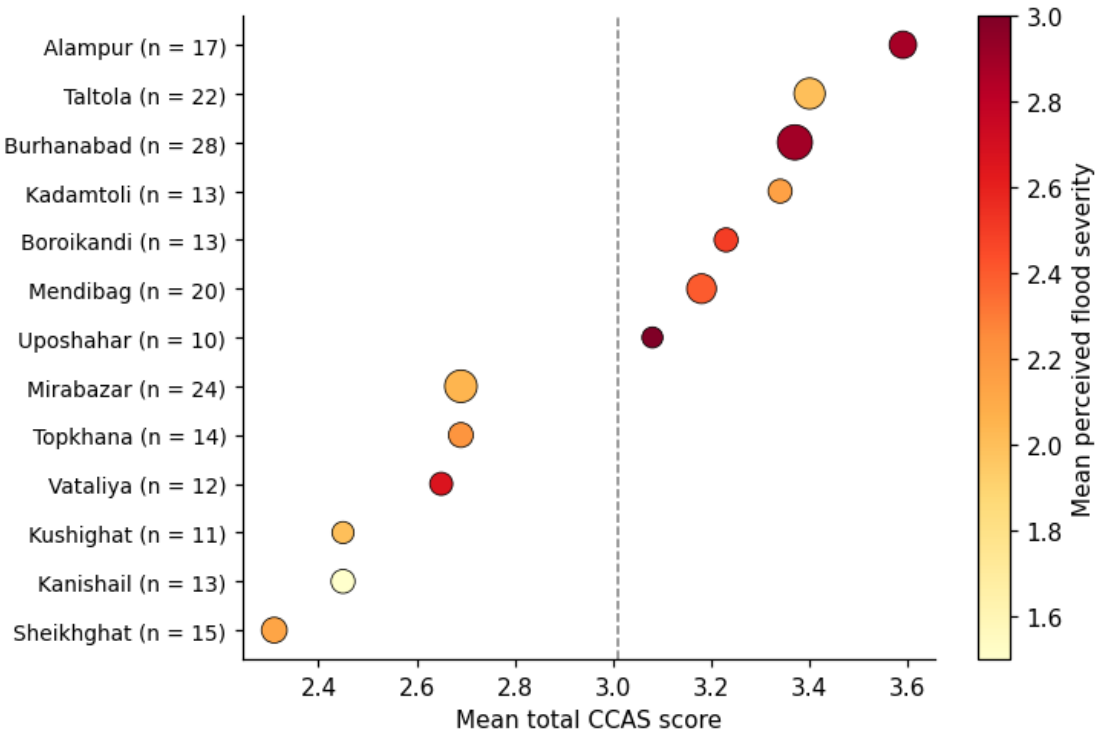
```
Fig2     174.0 x  91.4 mm  2055 x 1080 px  RGB
```



1.13.4 Fig. 3. Mean climate anxiety by neighbourhood, coloured by mean perceived flood severity

[21]:
```
plot4 = tab6.sort_values('mean')
fig, ax = plt.subplots(figsize=(W2, 4.6))
sc = ax.scatter(plot4['mean'], np.arange(len(plot4)), s=plot4['n'] * 8,
                c=plot4['mean_severity'], cmap='YlOrRd', edgecolor='black',␣
 ↪linewidth=.5,
                zorder=3)
ax.axvline(d['ccas_total'].mean(), color='grey', ls='--', lw=1)
ax.set_yticks(np.arange(len(plot4)))
ax.set_yticklabels(['%s (n = %d)' % (a, n) for a, n in zip(plot4.index,␣
 ↪plot4['n'])], fontsize=9)
ax.set_xlabel('Mean total CCAS score')
fig.colorbar(sc, label='Mean perceived flood severity')
save_fig(fig, 'Fig3')
plt.show()
```

```
Fig3     174.0 x 116.8 mm  2055 x 1380 px  RGB
```

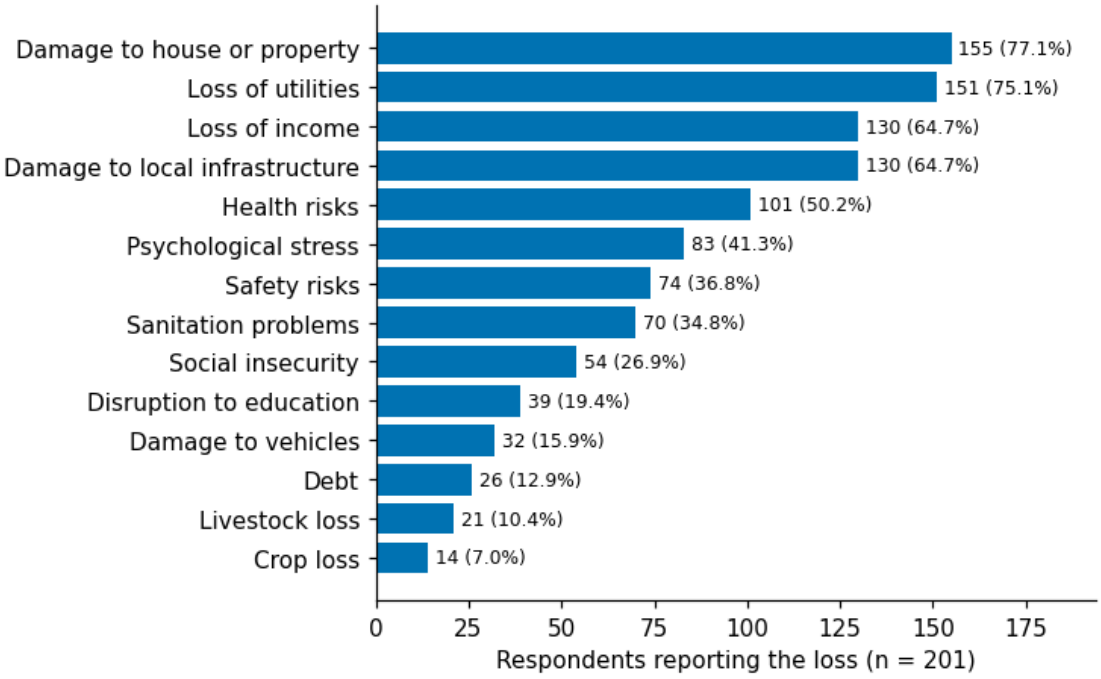


1.13.5 Fig. S1. Types of flood-related loss reported, ordered by frequency

Moved to Online Resource 8 to stay inside the journal's word budget.

[22]:
```
plotS1 = loss_tab.sort_values('n')
fig, ax = plt.subplots(figsize=(W2, 4.3))
ax.barh(plotS1.index, plotS1['n'], color=OKABE[0])
for y, (cnt, pct) in enumerate(zip(plotS1['n'], plotS1['% of 201'])):
    ax.text(cnt + 2, y, '%d (%.1f%%)' % (cnt, pct), va='center', fontsize=8)
ax.set_xlim(0, plotS1['n'].max() * 1.25)
ax.set_xlabel('Respondents reporting the loss (n = %d)' % n_loss)
save_fig(fig, 'FigS1')
plt.show()
```

```
FigS1    174.0 x 109.2 mm  2055 x 1290 px  RGB
```



1.14 14. Summary check

A last pass that prints the headline numbers next to the values printed in the manuscript, so that any future change to the data file shows up immediately.

[23]:
```
checks = [
    ('Eligible adults', len(raw), 212),
    ('Regression N', N3, 196),
    ("Cronbach's alpha, full scale", round(cronbach_alpha(items), 2), 0.88),
    ('Total CCAS mean', round(d['ccas_total'].mean(), 2), 3.01),
    ('Total CCAS SD', round(d['ccas_total'].std(ddof=1), 2), 0.72),
    ('High anxiety, n', int((d['band'] == 'High').sum()), 48),
    ('KMO', round(kmo, 2), 0.88),
    ('Bartlett chi-square', round(chi_sq, 1), 1425.0),
    ('Variance, two factors (%)', round(eig[:2].sum() / p * 100, 1), 57.4),
    ('Block 3 adjusted R2', round(adj_3, 2), 0.27),
    ('Flood severity beta', round(tbl3.loc['Flood severity', 'beta'], 2), 0.26),
    ('Number of losses beta', round(tbl3.loc['Number of losses', 'beta'], 2), 0.
 ↪20),
    ('Authorities respond beta', round(tbl3.loc['Authorities respond', 'beta'],␣
 ↪2), 0.17),
    ('Neighbourhood F', round(F_obs, 2), 7.96),
    ('ICC', round(icc, 2), 0.30),
]
out = pd.DataFrame(checks, columns=['statistic', 'computed', 'manuscript'])
```

```
out['match'] = np.where(np.isclose(out['computed'].astype(float),␣
 ↪out['manuscript'].astype(float)), 'ok', 'DIFFERS')
print(out.to_string(index=False))
print()
print('All %d headline statistics match the manuscript: %s'
      % (len(out), (out['match'] == 'ok').all()))
```

```
                statistic  computed  manuscript match
            Eligible adults    212.00      212.00    ok
               Regression N    196.00      196.00    ok
Cronbach's alpha, full scale      0.88        0.88    ok
            Total CCAS mean      3.01        3.01    ok
              Total CCAS SD      0.72        0.72    ok
            High anxiety, n     48.00       48.00    ok
                        KMO      0.88        0.88    ok
         Bartlett chi-square   1425.00     1425.00    ok
   Variance, two factors (%)     57.40       57.40    ok
         Block 3 adjusted R2      0.27        0.27    ok
         Flood severity beta      0.26        0.26    ok
       Number of losses beta      0.20        0.20    ok
    Authorities respond beta      0.17        0.17    ok
            Neighbourhood F      7.96        7.96    ok
                        ICC      0.30        0.30    ok

All 15 headline statistics match the manuscript: True
```